

VNU-HCM University of Information Technology

ASSIGNMENT 1

By Chu Cuong - 24520236

Email: cuongchu200@gmail.com
3-13-2025

MỤC LỤC

Lời mở đầu	2
1. VQ44_FLOWERS	3
2. Point2D	5
3. VS14_Gifts.....	7
4. PasswordStrength.....	9
5. CaesarCipher	13
6. ReversingEncryption.....	15
7. Messages	17
8. Binary Search 1	19
9. Binary Search 2	22
10. Linear Search 1	25
11. Linear Search 2	27
12. Linear Search 3	29
13. Linear Search 5	31
14. VW05p_Enrichement.....	33

For more details, visit github: [Link](#)

Lời mở đầu

Thuật toán tìm kiếm và sắp xếp là một trong những chủ đề quan trọng trong khoa học máy tính, đóng vai trò nền tảng trong việc xử lý và tổ chức dữ liệu.

Trong bài báo cáo này, tôi sẽ trình bày bài tập lớn Assignment1 với 14 bài tập liên quan đến các thuật toán tìm kiếm và sắp xếp của môn Cấu trúc dữ liệu và giải thuật.

Với mục đích ôn tập và củng cố kiến thức về lập trình và tư duy thuật toán, những bài tập được giao ở mức độ cơ bản giúp cho ta bước đầu dễ dàng tiếp cận hơn với môn học này.

[IT003.P21.CTTN.2] Assignment1 (Basic) 

14 problems with a total score of 1400

#	PROBLEM	SCORE
1	VQ44_FLOWERS	100
2	Point2D	100
3	VS14_Gifts	100
4	PasswordStrength	100
5	CaesarCipher	100
6	ReversingEncryption	100
7	Messages	100
8	Binary_Search 1	100
9	Binary_Search 2	100
10	Linear_Search 1	100
11	Linear_Search 2	100
12	Linear_Search 3	100
13	Linear_Search 5	100
14	VW05p_Enrichement	100

1. VQ44_FLOWERS

➤ Đề bài:

VQ44. ĐÈN HOA

Tên chương trình: FLOWERS.???

Để chuẩn bị một ngày lễ lớn lần thứ k thành phố quyết định dùng đèn LED kết thành k bông hoa trang trí dọc đường phố chính, mỗi bông hoa được kết từ một loại bóng LED cùng màu. Để có hiệu ứng ánh sáng tốt nhất Công ty Chiếu sáng được yêu cầu tạo ra được càng nhiều hoa khác màu càng tốt. Trong kho của Công ty có n bộ đèn LED mỗi bộ lắp được một đèn và bộ thứ i có màu a_i .

Hãy chỉ ra k màu của các bộ đèn cần chọn sao cho số lượng hoa khác màu lắp được là nhiều nhất. Nếu có nhiều cách khác nhau thì đưa ra cách tùy chọn.

Dữ liệu: Vào từ file văn bản FLOWERS.INP:

- ✚ Dòng đầu tiên chứa 2 số nguyên n và k ($1 \leq k \leq n \leq 10^5$),
- ✚ Dòng thứ 2 chứa n số nguyên $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9, i = 1 \div n$).

Kết quả: Đưa ra file văn bản FLOWERS.OUT trên một dòng k số nguyên – màu của các đèn được chọn.

Ví dụ:

FLOWERS.INP	FLOWERS.OUT
10 4	1 2 8 8
8 8 8 8 8 8 8 8 2 1	



➤ Ý tưởng:

Đề bài yêu cầu ta chọn k màu (k số) sao cho số lượng hoa khác màu (số khác nhau) là nhiều nhất. Theo đó, ta nghĩ ngay tới việc chọn ra tất cả các số khác nhau trước rồi thêm những số còn lại được nhập vào mà chưa được chọn (nếu chưa đủ k số).

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n, k;
    cin >> n >> k;
    vector<int> a, res;

    for (int i = 0; i < n; i++){
        int temp;
        cin >> temp;
        a.push_back(temp);
    }
    // Sắp xếp mảng theo thứ tự tăng dần
    sort(a.begin(), a.end());
    // Khai báo vector after để đánh dấu biến đã đưa vào res
    vector<int> after = a;
    // Duyệt từ đầu tới cuối vector a để chọn các phần tử khác nhau
    res.push_back(a[0]);
    after[0] = 0;
    int temp1 = 0, temp2 = 0;
    for (int i = 1; i < n; i++){
        if (res[temp1] != a[i] && res.size() < k){
            res.push_back(a[i]);
            after[i] = 0;
            temp1++;
        }
    }

    // Duyệt qua vector after và thêm vào các phần tử còn lại cho đủ
    while (res.size() < k){
        if (after[temp2] != 0){
            res.push_back(after[temp2]);
            after[temp2] = 0;
        }
        temp2++;
    }

    for (int j : res)
        cout << j << " ";
}
```

2. Point2D

➤ Đề bài:

Tập điểm:

Cho một tập N điểm (x,y) trên mặt phẳng Oxy. Bạn hãy sắp xếp và in ra các điểm tăng dần theo x , nếu x bằng nhau thì sắp xếp các điểm giảm dần theo y .

Input:

- Dòng đầu tiên chứa số nguyên N , ($1 \leq N \leq 10^6$) là số lượng điểm trên mặt phẳng.
- N dòng tiếp theo gồm 2 số nguyên x, y ($-10^9 \leq x, y \leq 10^9$) là tọa độ của các điểm trên mặt phẳng.

Output: Các điểm đã được sắp xếp, mỗi điểm được in trên một hàng.

Ví dụ:

Input	Output
5	1 3
1 2	1 2
1 3	2 9
2 4	2 4
2 9	4 1
4 1	

Lưu ý: Sử dụng '\n' để xuống dòng thay vì endl để tránh bị **Time Limit Exceeded (TLE)**

➤ Ý tưởng:

Thiết lập một vector chứa cặp số nguyên để lưu trữ điểm rồi sử dụng hàm sort của C++ để đặt hàm sắp xếp tương ứng với số nguyên x,y thỏa mãn yêu cầu đề.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second

bool comp(pair<int, int> a, pair<int, int> b)
{
    // Nếu hoành độ khác nhau thì sắp cho nó tăng dần
    if (a.fi != b.fi)
        return a.fi < b.fi;
    // Nếu hoành độ bằng nhau thì sắp tung độ giảm dần
    return a.se > b.se;
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;

    vector<pair<int, int>> v;
    for (int i = 0; i < n; i++)
    {
        int temp1, temp2;
        cin >> temp1 >> temp2;
        v.push_back({temp1, temp2});
    }
    sort(v.begin(), v.end(), comp);
    for (auto i : v)
        cout << i.fi << " " << i.se << '\n';
    return 0;
}
```

3. VS14_Gifts

➤ Đề bài:

QUÀ TẶNG

Tên chương trình: GIFTS.CPP

Nhân dịp năm mới Steve quyết định mua tặng 2 người bạn thân của mình mỗi người một món quà. Trong cửa hàng lưu niệm có n mặt hàng khác nhau, mặt hàng thứ i có giá a_i , $i = 1 \div n$. Với tổng số tiền trong túi là x , Steve quyết định sẽ mua 2 món quà khác nhau có tổng giá trị lớn nhất và tất nhiên – không vượt quá khả năng chi trả của mình.



Ví dụ, có 6 mặt hàng với giá nêu ở trên và số tiền có thể chi tối đa là 18, Steve sẽ chọn các món quà thứ nhất và thứ ba. Tổng số tiền cần chi sẽ là $5 + 10 = 15$.

Hãy xác định tổng số tiền Steve cần chi trả.

Dữ liệu: Vào từ thiết bị nhập chuẩn:

- ✚ Dòng đầu tiên chứa một số nguyên n và x ($2 \leq n \leq 10^5$, $2 \leq x \leq 10^9$),
- ✚ Dòng thứ 2 chứa n số nguyên $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$, $i = 1 \div n$).

Kết quả: Đưa ra thiết bị xuất chuẩn một số nguyên – số tiền cần chi trả.

Ví dụ:

INPUT	OUTPUT
6 18	15
5 3 10 2 4 9	

➤ Ý tưởng:

Sử dụng kỹ thuật hai con trỏ để duyệt qua mảng từ hai đầu để tìm ra tổng của hai số được con trỏ trỏ tới với độ phức tạp tối ưu.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n, k;
    cin >> n >> k;
    vector<int> v;
    for (int i = 0; i < n; i++){
        int temp;
        cin >> temp;
        v.push_back(temp);
    }
    sort(v.begin(), v.end());

    int temp = 0, i = 0, j = v.size() - 1;
    // Kỹ thuật hai con trỏ giúp chỉ duyệt qua một lần mảng
    while (i < j)
    {
        int sum = v[i] + v[j];
        // Nếu tổng nhỏ hơn hoặc bằng k, cập nhật temp và con trỏ đầu
        if (sum <= k)
        {
            temp = max(temp, sum);
            i++;
        }
        // Nếu không, cập nhật con trỏ cuối
        else j--;
    }
    cout << temp;
    return 0;
}
```

4. PasswordStrength

➤ Đề bài:

Hiện nay, đã có nhiều trường hợp các tài khoản cá nhân của người dùng bị tấn công bởi các hacker. Đôi khi chỉ cần bạn tham gia một trò chơi bất kỳ trên mạng xã hội và đăng nhập vào trò chơi đó bằng thông tin từ tài khoản cá nhân là đã cấp quyền tiết lộ thông tin.

Do đó, nếu người dùng có mật khẩu lỏng lẻo thì việc hacker xâm nhập và lấy cắp thông tin là càng có thể xảy ra. Hiểu được vấn đề an ninh mạng, nhiều nhà cung cấp dịch vụ đã và đang yêu cầu khách hàng của mình thay đổi mật khẩu thường xuyên với các chuẩn thiết lập mật khẩu như phải đủ độ dài ký tự, phải chứa ký tự in hoa, ký tự đặc biệt, con số...

Vì vậy, việc xây dựng một mật khẩu mạnh để bảo vệ an toàn cho các tài khoản cá nhân hiện nay là vô cùng quan trọng.



Trên thực tế có rất nhiều công thức để xác định tính mạnh yếu của mật khẩu, nhưng một trong những công thức cơ bản là:

$$\text{score} = \text{BaseScore} + (\text{Num_Excess} * \text{Bonus_Excess}) + \text{Num_Upper} * \text{Bonus_Upper} \\ + (\text{Num_Numbers} * \text{Bonus_Numbers}) + (\text{Num_Symbols} * \text{Bonus_Symbols}) + \\ \text{Bonus_Combo} + \text{Bonus_FlatLower} + \text{Bonus_FlatNumber}$$

Trong đó:

Biến	Giá trị	Điều kiện
BaseScore	40	
Bonus_Excess	3	
Bonus_Upper	4	
Bonus_Number	5	
Bonus_Symbols	5	

Bonus_Combo	25	Nếu có chữ cái in hoa, kí tự đặc biệt và chữ số trong mật khẩu.
	15	Nếu mật khẩu có chứa các kí tự thuộc 2 trong 3 tập (chữ cái in hoa, kí tự đặc biệt, chữ số)
	0	Ngược lại
Bonus_FlatLower	-15	Nếu mật khẩu chứa toàn kí tự là chữ cái thường
	0	Ngược lại
Bonus_FlatNumber	-35	Nếu mật khẩu đều là kí tự chữ số
	0	Ngược lại
Number_Excess	Hiệu giữa độ dài mật khẩu và 8	
Number_Upper	Số lượng chữ cái in hoa trong mật khẩu	
Number_Numbers	Số lượng chữ số trong mật khẩu	
Number_Symbols	Số lượng kí tự đặc biệt trong mật khẩu	

Lưu ý:

- + Tập kí tự đặc biệt là gồm các kí tự [! , @ , # , \$, % , ^ , & , * , ? , _ , ~]
- + Nếu mật khẩu chứa các kí tự có độ dài < 8 hoặc có một trong các dấu [. , \ , / , ' , ' , '] thì là một mật khẩu không hợp lệ .
- + Nếu **score < 50** là mật khẩu yếu, **50 ≤ score < 75** là mật khẩu vừa, **75 ≤ score < 100** là mật khẩu mạnh, **score ≥ 100** là mật khẩu rất mạnh.

Bạn hãy viết một chương trình để xác định xem độ mạnh của mật khẩu?

Dữ liệu: Vào từ thiết bị nhập chuẩn gồm một dòng chứa xâu s là mật khẩu cần kiểm tra ($1 \leq |s| \leq 100$).

Kết quả: Đưa ra thiết bị xuất chuẩn là các đánh giá về mật khẩu ('KhongHopLe', 'Yeu', 'Vua', 'Manh', 'RatManh') .

Ví dụ:

INPUT		OUTPUT
Alice@2503		RatManh

➤ **Ý tưởng:**

Khai báo các biến và xét các trường hợp thêm của biến (nếu có) và sử dụng công thức để xác định tính mạnh yếu của mật khẩu và in ra màn hình.

➤ **Script:**

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second
const signed BaseScore = 40;
const signed Bonus_Excess = 3;
const signed Bonus_Upper = 4;
const signed Bonus_Number = 5;
const signed Bonus_Symbols = 5;
signed Bonus_Combo;
signed Bonus_FlatNumber;
signed Bonus_FlatLower;
signed Number_Upper;
signed Number_Lower;
signed Number_Numbers;
signed Number_Symbols;
signed Number_dif;

bool issymbol(char c){
    vector<char> symbol = {'!', '@', '#', '$', '%', '^', '&', '*', '?',
'_', '~'};
    for (char p : symbol)
    {
        if (c == p)
            return true;
    }
    return false;
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    string s;
    cin >> s;
    const signed n = s.size();
    signed Number_Excess = n - 8;

    if (n < 8){
        cout << "KhongHopLe";
        return 0;
    }
```

```

for (signed i = 0; i < n; i++){
    if (isupper(s[i]))
        Number_Upper++;
    else if (islower(s[i]))
        Number_Lower++;
    else if (isdigit(s[i]))
        Number_Numbers++;
    else if (issymbol(s[i]))
        Number_Symbols++;
    else{
        cout << "KhongHopLe";
        return 0;
    }
}

// Đặt số Bonus_Combo theo đề
if (Number_Upper > 0 && Number_Symbols > 0 && Number_Numbers > 0)
    Bonus_Combo = 25;
else if ((Number_Upper > 0 && Number_Symbols > 0) || (Number_Symbols >
0 && Number_Numbers > 0) || (Number_Upper > 0 && Number_Numbers > 0))
    Bonus_Combo = 15;
else if (Number_Upper == 0 && Number_Symbols == 0 && Number_Numbers ==
0) Bonus_Combo = 0;

// Đặt số Bonus_FlatLower theo đề
if (Number_Lower == n) Bonus_FlatLower = -15;
else Bonus_FlatLower = 0;

// Đặt số Bonus_FlatNumber theo đề
if (Number_Numbers == n) Bonus_FlatNumber = -35;
else Bonus_FlatLower = 0;

int point = BaseScore + Number_Excess * Bonus_Excess + Number_Upper *
Bonus_Upper + Number_Numbers * Bonus_Number + Number_Symbols *
Bonus_Symbols + Bonus_Combo + Bonus_FlatLower + Bonus_FlatNumber;
if (point < 50) cout << "Yeu";
else if (point < 75) cout << "Vua";
else if (point < 100) cout << "Manh";
else cout << "RatManh";
return 0;
}

```

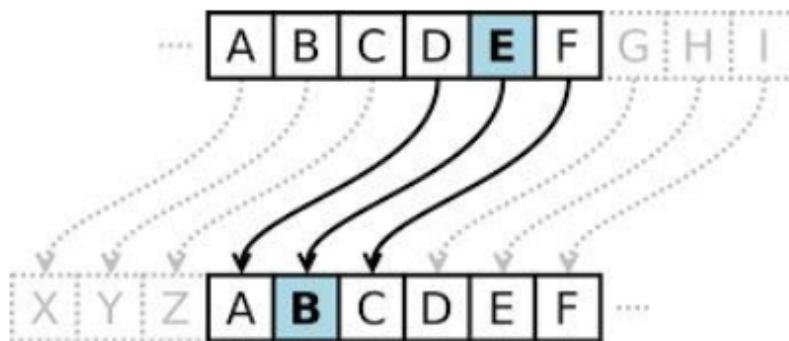
5. CaesarCipher

➤ Đề bài:

Trong mật mã học, **mật mã Caesar**, (còn được gọi là **Caesar code** hoặc **Caesar shift**), là một trong những kỹ thuật mã hóa đơn giản nhất và được biết đến rộng rãi nhất. Nó là một loại mật mã thay thế, trong đó mỗi chữ cái trong **plaintext** được thay thế bằng một bức thư một số số cố định (**key**) của các vị trí lên/xuống bảng chữ cái. Phương pháp này được đặt theo tên của Julius Caesar, người sử dụng nó trong thư riêng của ông.

Mã hóa cũng có thể được biểu diễn thông qua số học mô-đun bằng cách gán các ký tự bằng các con số, theo tuần tự, **A = 0, B = 1, ..., Z = 25**. Mã hóa một chữ cái x bằng phép dịch chuyển k vị trí (**key**) có thể mô tả bằng biểu thức toán học sau:

$$E(x) = (x + k) \bmod 26$$



Dữ liệu:

- Dòng đầu tiên số tự nhiên K cho biết độ dịch chuyển ($0 \leq K \leq 10^9$)
- Dòng tiếp theo là một chuỗi ký tự hoa có độ dài bé hơn bằng 10000 ký tự.

Kết quả:

Kết quả là chuỗi ký tự (**cipher text**) cho biết kết quả của phép mã hóa Caesar.

Ví dụ:

INPUT	OUTPUT
10 THE END	DRO OXN

➤ **Ý tưởng:**

Đơn giản, ta chỉ cần viết lệnh nhập chuỗi và duyệt xâu từ đầu tới cuối rồi dịch chuyển k vị trí của từng phần tử sau đó nối lại và ra được xâu được mã hóa.

➤ **Script:**

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second
const int N = 1e6 + 9;
const int N2 = N * 10;
const int mod = 1e9 + 7;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int k;
    string s;

    cin >> k;
    // Bỏ qua '\n'
    cin.ignore();
    // Lấy cả dấu cách
    getline(cin, s);

    for (int i = 0; i < s.length(); i++){
        // Khi gặp dấu cách, in ra khoảng trắng rồi bỏ qua
        if (s[i] == ' '){
            cout << " ";
            continue;
        }
        // Dùng thuật  $E(x) = (x+k) \bmod 26$ 
        s[i] = (s[i] - 'A' + k) % 26 + 'A';
        cout << s[i];
    }
}
```

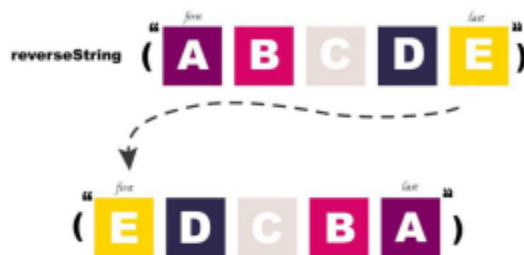
6. Reversing Encryption

➤ Đề bài:

Một chuỗi s (**plaintext**) có độ dài n có thể được mã hóa bằng thuật toán sau:

- + Duyệt trên tất cả các ước của n theo thứ tự giảm dần từ n đến 1.
- + Với mỗi ước số d , đảo ngược chuỗi con $s[1 \dots d]$ (tức là chuỗi con bắt đầu ở vị trí 1 và kết thúc ở vị trí d).

Ví dụ, thuật toán trên được áp dụng cho chuỗi $s = \text{"ATTN2020"}$ dẫn đến những thay đổi sau:
 $\text{"0202NNTTA"} \rightarrow \text{"2020NNTTA"} \rightarrow \text{"0220NNTTA"} \rightarrow \text{"0220NTTA"}$ (rõ ràng là thao tác đảo ngược cuối cùng không thay đổi chuỗi vì $d = 1$).



Bây giờ, bạn sẽ được cung cấp chuỗi đã được mã hóa t (**ciphertext**).

Nhiệm vụ của bạn là giải mã chuỗi t này, tức là, để tìm một chuỗi s sao cho sau khi thực hiện thuật toán trên cho kết quả là chuỗi t . Luôn đảm bảo chuỗi s này luôn tồn tại và là duy nhất.

Dữ Liệu:

Đầu vào là chuỗi t là **ciphertext** gồm các chữ cái và chữ số ($|t| \leq 300$).

Kết quả:

Kết quả là chuỗi ký tự ban đầu (**plaintext**)

Ví dụ:

INPUT		OUTPUT
0220NTTA		ATTN2020

➤ Ý tưởng:

Đầu tiên, ta tìm tất cả các ước của n rồi đảo ngược lại theo thứ tự tăng dần từ 1 đến n để đảo ngược quá trình mã hóa của thuật toán trên.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    string s;
    cin >> s;
    const int n = s.size();
    vector<int> v;

    // Tìm các ước của n
    for (int i = 1; i <= sqrt(n); i++){
        if (i * i == n) v.push_back(i);
        else if (n % i == 0){
            v.push_back(i);
            v.push_back(n / i);
        }
    }
    sort(v.begin(), v.end());
    // Đảo ngược chuỗi con s[1...d] với thứ tự tăng dần từ 1 tới n
    for (int i : v){
        reverse(s.begin(), s.begin() + i);
    }
    cout << s;
    return 0;
}
```

7. Messages

➤ Đề bài:

Thầy Sơn cần gửi một văn bản quan trọng tới một đối tác của mình. Văn bản là một xâu S các chữ cái la tinh in thường. Để bảo mật nội dung văn bản, thầy Sơn gửi 2 bức thư. Bức thư thứ nhất là phần đầu S_b của xâu S , bức thư thứ 2 là phần cuối S_e của S . Hai bức thư S_b và S_e đảm bảo đầy đủ nội dung của S , tuy nhiên có thể một phần cuối của S_b có thể được viết lặp lại trong phần đầu của S_e , song số kí tự được viết lặp lại không biết trước.



Yêu cầu: Cho hai xâu S_b và S_e , hãy xác định một xâu S có thể là nội dung của bức thư sao cho độ dài của xâu S là ngắn nhất.

Dữ liệu:

Dòng đầu chứa xâu S_b , dòng thứ hai chứa xâu S_e . Mỗi xâu có độ dài không quá 500, gồm các chữ cái la-tinh thường

Kết quả:

Dòng đầu tiên là độ dài của xâu S ban đầu. Dòng thứ hai xuất ra xâu S .

Ví dụ:

INPUT		OUTPUT
ATTN2020 2020K15		11 ATTN2020K15

➤ **Ý tưởng:**

Lần lượt lấy chuỗi con tính từ cuối của Sb so sánh với chuỗi con từ đầu của Se và tìm ra chuỗi trùng. Đầu tiên, giả sử chuỗi con từ Sb là chính nó rồi so sánh với chuỗi có cùng độ dài tính từ đầu của Se, dần dần giảm độ dài của chuỗi này đến khi tìm được chuỗi trùng cần tìm.

➤ **Script:**

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    string Sb, Se;
    cin >> Sb >> Se;
    string s1, s2, res;

    int i;
    for (i = 0; i < Sb.size(); i++)
    {
        s1 = Sb.substr(i, Sb.size() - i);
        s2 = Se.substr(0, Sb.size() - i);
        if (s1 == s2)
            break;
    }
    res = Sb.substr(0, i) + Se;
    cout << res.size() << endl << res;
    return 0;
}
```

8. Binary Search 1

➤ Đề bài: Tìm kiếm nhị phân

Bài toán:

Thuật toán tìm kiếm nhị phân là một thuật toán tìm kiếm đơn giản nhưng có tốc độ tìm kiếm cực kỳ nhanh. Ý tưởng của thuật toán là: với một mảng N phần tử đã được sắp xếp (tăng dần/hoặc giảm dần tùy theo người ra đề) và số nguyên x cần tìm, ta sẽ thực hiện tìm kiếm ở miền có khả năng xuất hiện x sau mỗi lần lặp.

Yêu cầu: Hãy cài đặt thuật toán tìm kiếm nhị phân để tìm vị trí của phần tử x trong mảng có N phần tử và đếm số lần duyệt qua các phần tử.

Input :

- Dòng đầu tiên là số nguyên N dương ($0 < N < 32000$)
- Dòng tiếp theo chứa N số nguyên A_i là các phần tử của mảng đã được sắp xếp tăng dần và không trùng nhau
- Dòng cuối cùng là số nguyên x cần tìm

Output:

- Dòng đầu tiên là vị trí của x được tìm thấy trong mảng. Nếu không tìm thấy thì xuất ra -1
- Dòng tiếp theo là số lần duyệt qua các phần tử để tìm được x . Nếu không tìm thấy thì không cần xuất ra dòng này

Ví dụ:

INPUT	OUTPUT	Giải thích
8 1 3 4 5 10 12 15 20 4	2 3	- 4 được tìm thấy ở vị trí 2 - để tìm được 3 cần phải duyệt qua các phần tử ở vị trí 3, 1 và 2.
4 1 2 3 4 -9	-1	- không tìm thấy -9 trong mảng

Lưu ý: Số lần duyệt để tìm x trong bài sẽ dựa trên pseudo code sau đây:

```
function binary_search(A, n, T) is  
  L := 0  
  R := n - 1  
  while L ≤ R do  
    m := floor((L + R) / 2)  
    if A[m] < T then  
      L := m + 1  
    else if A[m] > T then  
      R := m - 1  
    else:  
      return m  
  return unsuccessful
```

➤ **Ý tưởng:**

Dựa vào pseudo code, ta có thể viết được hàm tìm kiếm nhị phân. Đầu tiên, ta lấy phần tử chính giữa của mảng và so sánh nó với số nguyên cần tìm, nếu lớn hơn thì ta xét chuỗi từ đầu tới vị trí đó, ngược lại thì lấy chuỗi từ vị trí đó tới cuối. Cứ tiếp tục như vậy tới khi tìm thấy số nguyên cần tìm.

Số lần duyệt được tính sau mỗi lần cập nhật phần tử chính giữa của mảng đang tìm kiếm. Độ phức tạp của thuật toán: $O(\log n)$

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second

// Hàm vừa tìm số nguyên vừa đếm số lần duyệt
pair<int, int> binary_search(vector<int> a, int n, int k){
    int left = 0, right = n - 1, count = 0;
    while (left <= right)
        // Biến đếm để in số lần duyệt
        count++;
        int mid = (left + right) / 2;
        if (a[mid] == k) return {mid, count};
        else if (a[mid] < k) left = mid + 1;
        else right = mid - 1;
    }
    return {-1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n, k;
    vector<int> a;
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        int temp;
        cin >> temp;
        a.push_back(temp);
    }
    cin >> k;

    pair<int, int> res = binary_search(a, n, k);
    if (res.fi == -1)
        cout << -1 << endl;
    else
        cout << res.fi << endl
            << res.se << endl;
}
```

9. Binary Search 2

➤ Đề bài: Tìm kiếm nhị phân 2

Bài toán:

Thuật toán tìm kiếm nhị phân là một thuật toán tìm kiếm đơn giản nhưng có tốc độ tìm kiếm cực kỳ nhanh. Tuy nhiên, đa phần các hướng dẫn và khóa học trên mạng đều hướng dẫn người học cài đặt trên mảng số nguyên trong khi trong thực tế chỉ cần mảng có thể sắp xếp được thì ra có thể áp dụng thuật toán tìm kiếm nhị phân.

Yêu cầu: Hãy cài đặt thuật toán tìm kiếm nhị phân trên mảng chuỗi có N phần tử.

Input :

- Dòng đầu tiên là số nguyên N dương ($0 < N < 2000$)
- N dòng tiếp theo chứa các phần tử A_i trong mảng, mỗi phần tử là một chuỗi với độ dài không quá 10. Các phần tử này đã được sắp xếp theo thứ tự bảng chữ cái
- Dòng cuối cùng là chuỗi x cần tìm

Output:

- Dòng đầu tiên là vị trí của x được tìm thấy trong mảng. Nếu không tìm thấy thì xuất ra -1
- Dòng tiếp theo là số lần duyệt qua các phần tử để tìm được x. Nếu không tìm thấy thì không cần xuất ra dòng này

Ví dụ:

INPUT	OUTPUT	Giải thích
8 2024 Cau Giai Lieu Thuat Va du truc Va	5 2	- 'Va' được tìm thấy ở vị trí 5 - để tìm được 'Va' cần phải duyệt qua 2 vị trí là 3 và 5
4 2024 Happy New Year 2023	-1	- không tìm thấy

Giải thuật:

Cài đặt giải thuật binary search

➤ **Ý tưởng:**

Giống bài trước chỉ khác ở việc thay đổi số bằng xâu ký tự.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second

pair<int, int> binary_search(vector<string> a, int n, string k){
    int left = 0, right = n - 1, count = 0;
    while (left <= right){
        // Biến đếm để in số lần duyệt
        count++;
        int mid = (left + right) / 2;
        if (a[mid] == k)
            return {mid, count};
        else if (a[mid] < k)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return {-1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    string k;
    vector<string> a;
    cin >> n;
    for (int i = 0; i < n; i++)
        string temp;
        cin >> temp;
        a.push_back(temp);
    }
    cin >> k;

    pair<int, int> res = binary_search(a, n, k);
    if (res.fi == -1)
        cout << -1 << endl;
    else
        cout << res.fi << endl
            << res.se << endl;
}
```

10. Linear Search 1

➤ Đề bài: Tìm kiếm tuyến tính

Bài toán:

Thuật toán tìm kiếm tuyến tính là một trong những thuật toán tìm kiếm đơn giản và dễ dàng cài đặt nhất. Ý tưởng của thuật toán là: với một mảng cho trước có N phần tử và phần tử cần tìm x, duyệt từ đầu mảng đến cuối mảng cho đến khi tìm được phần tử x.

Yêu cầu: Hãy cài đặt thuật toán tìm kiếm tuyến tính để tìm vị trí của x đầu tiên trong mảng có N phần tử và đếm số lần duyệt qua các phần tử.

Input :

- Dòng đầu tiên là số nguyên N dương ($0 < N < 10000$)
- Dòng tiếp theo chứa N số nguyên A_i là các phần tử của mảng
- Dòng cuối cùng là số nguyên x cần tìm

Output:

- Dòng đầu tiên là vị trí của x đầu tiên trong mảng nếu tìm được. Nếu không tìm thấy thì xuất ra -1
- Số lần duyệt qua các phần tử của mảng (từ đầu mảng đến cuối) để tìm được x
- Dòng thứ ba là vị trí của x đầu tiên trong mảng (đếm từ cuối đến đầu) nếu tìm được. Nếu không tìm thấy thì xuất ra -1
- Số lần duyệt qua các phần tử của mảng (từ cuối mảng đến đầu) để tìm được x
- Nếu như ở 2 dòng đầu tiên không thấy thì không cần xuất ra 2 dòng tiếp theo

Ví dụ:

INPUT	OUTPUT	Giải thích
5	1	• -4 ở vị trí 1 tính từ đầu mảng • phải duyệt qua 2 lần để tìm được -4 • -4 ở vị trí 3 tính từ cuối mảng • phải duyệt qua 4 lần từ cuối mảng để tìm được -4
1 -4 3 2 6	2	
-4	3	
	4	
4	-1	• Không có phần tử -9 trong mảng
1 2 3 4		
-9		

➤ Ý tưởng:

Đơn giản là duyệt từ đầu tới cuối mảng để tìm phần tử cần tìm đầu tiên

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int LinearSearch(vector<int> &a, int n, int k){
    // Duyệt từ đầu tới cuối mảng để tìm phần tử
    for (int i = 0; i < n; i++){
        if (a[i] == k) return i;
    }
    return -1;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n, k;
    vector<int> a;
    cin >> n;

    for (int i = 0; i < n; i++){
        int temp;
        cin >> temp;
        a.push_back(temp);
    }
    cin >> k;

    int res = LinearSearch(a, n, k);
    if (res == -1) cout << res;
    else{
        cout << res << endl;
        cout << res + 1 << endl;
        cout << n - 1 - res << endl;
        cout << n - res << endl;
    }
}
```

11. Linear Search 2

➤ Đề bài: Tìm kiếm tuyến tính 2

Bài toán:

Thuật toán tìm kiếm tuyến tính là một trong những thuật toán tìm kiếm đơn giản và dễ dàng cài đặt nhất. Ý tưởng của thuật toán là: với một mảng cho trước có N phần tử và phần tử cần tìm x, duyệt từ đầu mảng đến cuối mảng cho đến khi tìm được phần tử x.

Yêu cầu: Hãy cài đặt thuật toán tìm kiếm tuyến tính để tìm vị trí của tất cả x trong mảng có N phần tử và đếm số lần duyệt qua các phần tử.

Input :

- Dòng đầu tiên là số nguyên N dương ($0 < N < 10000$)
- Dòng tiếp theo chứa N số nguyên A_i là các phần tử của mảng
- Dòng cuối cùng là số nguyên x cần tìm

Output:

- Dòng đầu tiên là số lần x được tìm thấy trong mảng
- Mỗi dòng tiếp theo tương ứng với vị trí của x và số lần duyệt từ đầu mảng để tìm được x

Ví dụ:

INPUT	OUTPUT	Giải thích
8 1 3 -4 2 6 10 3 12 3	2 1 2 6 7	- 3 được tìm thấy ở 2 vị trí là 1 và 6, tương ứng phải duyệt 2 và 7 lần để tìm được
4 1 2 3 4 -9	0	- không tìm thấy -9 trong mảng

➤ **Ý tưởng:**

Sử dụng thuật toán tìm kiếm tuyến tính để duyệt từ đầu tới cuối mảng để tìm phần tử cần tìm và dùng vector để lưu trữ vị trí và số lần duyệt của từng phần tử đó.

➤ **Script:**

```
#include <bits/stdc++.h>
using namespace std;

int LinearSearch(vector<int> &a, int start, int n, int k){
    for (int i = start; i < n; i++){
        if (a[i] == k) return i;
    }
    return -1;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    vector<int> a;
    cin >> n;
    for (int i = 0; i < n; i++){
        int temp;
        cin >> temp;
        a.push_back(temp);
    }
    cin >> k;
    vector<int> res;
    int temp = LinearSearch(a, 0, n, k);
    if (temp == -1){
        cout << 0;
        exit(0);
    }
    res.push_back(temp);
    // Tìm tất cả các phần tử trùng còn lại bắt đầu từ vị trí trước
    while (res[res.size() - 1] != -1){
        int next = LinearSearch(a, res[res.size() - 1] + 1, n, k);
        res.push_back(next);
    }
    cout << res.size() - 1 << endl;
    for (int i = 0; i < res.size() - 1; i++){
        cout << res[i] << " " << res[i] + 1 << endl;
    }
}
```

12. Linear Search 3

➤ Đề bài: Find MEX

Bài toán:

Cho trước một mảng có N phần tử. Hãy in ra MEX_i với $0 \leq i < n$.

Biết rằng, MEX_i là số nguyên nhỏ nhất lớn hơn hoặc bằng 0 mà không xuất hiện trong từ đầu mảng cho đến phần tử i .

Input :

- Dòng đầu tiên là N ($1 \leq N \leq 2 \times 10^5$) thể hiện số phần tử của mảng
- Dòng tiếp theo chứa N số nguyên là các phần tử của mảng

Output:

- N số nguyên tương ứng với MEX_i

Ví dụ:

INPUT	OUTPUT	Giải thích
5 1 0 5 5 3	0 2 2 2 2	- Với $i = 0$, thì số nhỏ nhất chưa xuất hiện từ đầu mảng đến phần tử 0 là 0. - Với $i = 1$, thì số nhỏ nhất chưa xuất hiện từ đầu mảng đến phần tử 1 là 2. - Với $i = 2$, thì số nhỏ nhất chưa xuất hiện từ đầu mảng đến phần tử 2 là 2. - Với $i = 3$, thì số nhỏ nhất chưa xuất hiện từ đầu mảng đến phần tử 3 là 2. - Với $i = 4$, thì số nhỏ nhất chưa xuất hiện từ đầu mảng đến phần tử 4 là 2.

➤ Ý tưởng:

Ta sẽ duyệt qua mảng từ đầu đến cuối và lập một mảng ánh xạ với những phần tử đã đi qua, song song với đó so sánh MEX với những phần tử trong mảng ánh xạ đó để tìm ra MEX nhỏ nhất thỏa mãn qua từng vị trí.

Script:

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e6 + 9;

vector<int> LinearSearchMEX(vector<int> &a, int n)
{
    vector<int> res;
    bool b[N];
    int MEX = 0;
    for (int i = 0; i < n; i++)
    {
        // Lập một mảng ánh xạ với phần tử đã qua
        b[a[i]] = true;
        while (b[MEX])
        {
            // Tìm MEX nhỏ nhất mà không trùng với phần tử trước
            MEX++;
        }
        res.push_back(MEX);
    }
    return res;
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    vector<int> a;
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        int temp;
        cin >> temp;
        a.push_back(temp);
    }

    vector<int> res = LinearSearchMEX(a, n);
    for (int i : res)
    {
        cout << i << " ";
    }
}
```

13. Linear Search 5

➤ Đề bài: Max different

Bài toán:

Trọng số của một mảng được định nghĩa là sự khác biệt giữa phần tử lớn nhất và nhỏ nhất của mảng. Ví dụ: trọng số của mảng $[3, 7, 1, 2]$ là $(7 - 1) = 6$, trọng số của mảng $[2]$ là $(2 - 2) = 0$, trọng số của mảng trống là 0.

Cho một mảng A có độ dài N. Bạn phải chia các phần tử của A thành hai dãy con S1 và S2 (một trong S1, S2 có thể trống) sao cho:

- Mỗi phần tử của mảng A chỉ thuộc một trong hai dãy S1 hoặc S2.
- Tổng trọng số của hai dãy S1 và S2 là lớn nhất.

Bạn in ra đáp án lớn nhất có thể của tổng trọng số của hai dãy S1 và S2.

Input :

- Dòng đầu tiên chứa T là số lượng test. Mô tả các trường hợp thử nghiệm T như sau:
- Với mỗi test case:
 - Dòng đầu tiên chứa N là kích thước của mảng A.
 - Dòng thứ hai chứa N số nguyên $A[1], A[2], \dots, A[N]$ - biểu thị các phần tử của A.

Constraints:

- $1 \leq T \leq 10^4$
- $2 \leq N \leq 10^5$
- $1 \leq A[i] \leq 10^9$
- Tổng của N trên tất cả các test case không quá 2×10^5

Output:

- Đối với mỗi trường hợp thử nghiệm, hãy in tổng trọng số tối đa có thể có của hai dãy con.

Ví dụ:

INPUT	OUTPUT	Giải thích
2 3 1 2 3 5 4 8 1 10 2	2 15	- Một phép chia có thể là [1, 3], [2] Ở đây tổng trọng số là $= (3 - 1) + (2 - 2) = 2$ - Ở testcase số 2, có thể chia thành 2 dãy [4, 1, 10] và [8, 2] thì tổng trọng số là $(10 - 1) + (8 - 2) = 15$

➤ Ý tưởng:

Xét mảng đã được sắp xếp tăng dần, ta có thể thấy là tổng trọng số của hai dãy S1 và S2 là lớn nhất khi nó là hiệu của tổng hai số lớn nhất và tổng hai số nhỏ nhất.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int t, n;
    cin >> t;
    for (int i = 0; i < t; i++){
        cin >> n;
        int a[n];
        for (int j = 0; j < n; j++) cin >> a[j];
        sort(a, a + n);
        // Nếu mảng có 2 phần tử, đưa hết vào S1 vì nếu không thì TS = 0
        if (n == 2){
            cout << a[1] - a[0] << "\n";
            continue;
        }
        // Nếu mảng có 3 phần tử, đưa 2 số đầu cuối vào S1, TS của S2 = 0
        if (n == 3){
            cout << a[2] - a[0] << "\n";
            continue;
        }
        cout << (a[n - 1] - a[0]) + (a[n - 2] - a[1]) << "\n";
    }
    return 0;
}
```

14. VW05p_Enrichement

➤ Đề bài:

Các nhà địa chất phát hiện một khu mỏ quặng đất hiếm, một thứ rất cần thiết cho công nghiệp chế tạo thiết bị điện tử. Khu mỏ có hình chữ nhật kích thước $n \times m$ ô. Trữ lượng quặng ở ô (i, j) được đánh giá là

$$a_{i,j} (1 \leq i \leq n, 1 \leq j \leq m)$$

. Cần xây dựng một xí nghiệp làm giàu quặng trước khi đưa ra thị trường. Do điều kiện địa hình, xí nghiệp phải xây dựng ngay trên khu mỏ. Xí nghiệp chiếm diện tích 3×3 ô. Dĩ nhiên không thể khai thác quặng dưới nền của xí nghiệp, vì vậy người ta muốn tìm vị trí đặt xí nghiệp sao cho tổng trữ lượng phải để lại là ít nhất.

Hãy xác định tổng trữ lượng nhỏ nhất phải để lại.

Dữ liệu vào :

Dòng đầu tiên chứa 2 số nguyên n và m ($3 \leq n, m \leq 10^6; 1 \leq n \times m \leq 10^6$),

Dòng thứ i trong n dòng sau chứa m số nguyên

$$a_{i1}, a_{i2}, \dots, a_{im} \quad (0 \leq a_{i,j} \leq 10^5, 1 \leq j \leq m)$$

Kết quả: Đưa ra một số nguyên – tổng trữ lượng nhỏ nhất phải để lại.

Ví dụ:

Input	Output
5 7 10 2 3 7 10 4 8 3 2 1 9 6 2 1 0 3 6 7 8 9 10 5 4 3 0 2 1 8 9 2 3 10 6 4 8	27

➤ Ý tưởng:

Ta tính tổng tất cả các số trong ma trận 3×3 ô của khu mỏ $n \times m$ ô rồi sắp xếp mảng các số này và lấy số nhỏ nhất thì được kết quả thỏa mãn. Đầu tiên, viết hàm tính tổng các số trong ma trận 3×3 ô tính từ ô ở góc trên cùng bên trái của ma trận. Tiếp theo, duyệt 1 điểm chạy hết bảng $(n - 2) \times (m - 2)$ ô nằm góc trên bên trái của khu mỏ và dùng hàm tính tổng từ ô đó thì ta sẽ thu được tất cả các tổng của ma trận 3×3 ô của khu mỏ này. Cuối cùng lưu trữ nó vào vector và sắp xếp thì ta sẽ lấy được giá trị tổng nhỏ nhất cần tìm.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

// Hàm tính tổng các số trong ma trận 3x3 ô
int sum_matrix(vector<vector<int>> &a, int n, int m)
{
    int sum = 0;
    for (int k = n; k < n + 3; k++)
    {
        for (int l = m; l < m + 3; l++)
            sum += a[k][l];
    }
    return sum;
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> a(n, vector<int>(m));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            cin >> a[i][j];
        }
    }
    // Tính tất cả tổng các ma trận 3x3 ô
    vector<int> res;
    for (int i = 0; i < n - 2; i++)
    {
        for (int j = 0; j < m - 2; j++)
            res.push_back(sum_matrix(a, i, j));
    }
    // Sắp xếp và in kết quả
    sort(res.begin(), res.end());
    cout << res[0];
    return 0;
}
```